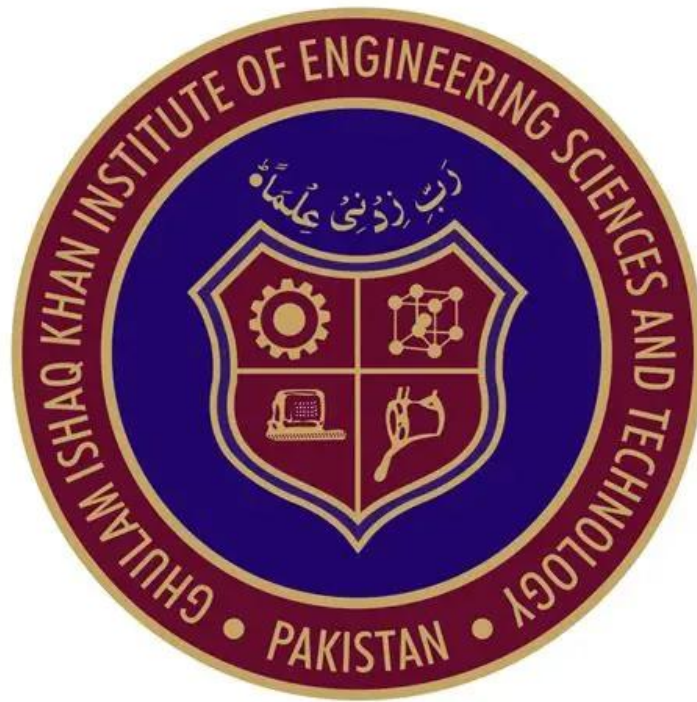




Secure QR-Based Attendance Management System

Final Project Report

Course: Secure Software Design (CY-321) Course

Instructor: Dr. Zubair Ahmed

Date: May 5, 2026

Group Members

Awais Khan	2023139
Moawiz-ur-Rehman	2023278
Zulqarnain Umar	2023556

Contents

1. Executive Summary.....	2
2. Introduction and Problem Statement	3
3. Proposed Solution: A Unified Secure Attendance System	3
4. Security Features and Threat Mitigation	4
5. Security Testing and Results	5
6. Risk Management Summary	6
7. Conclusion	6

Executive Summary

One big problem with old-school roll call? It wastes time, leaves gaps in security. Think about buddy punching — someone sneaks in for a friend who never shows up. A new fix comes into play: a system built tough, using QR codes to handle attendance right. Built like real software, not just a test model. Works reliably when it counts.

A fresh layer appears each time access is checked, using shifting QR patterns tied to encrypted signals. One-time keys tag every entry attempt alongside hidden markers from the user's machine. Hidden checks run beneath, stitching proof into each visit without gaps. Security builds quietly through overlapping steps that confirm presence reliably.

From the start, threats were mapped out with STRIDE to shape early design choices. Built-in safeguards took form through careful code reviews and strict rules during development. Instead of waiting until the end, checks happened at every stage using automated scans alongside live attack simulations. Each flaw found got fixed before moving forward. By launch, every serious risk had been addressed clearly. Strength came not from tools alone but consistent attention across phases. What emerged stood ready under pressure.

1. Introduction and Problem Statement

Some schools keep struggling to track who shows up each day. Paper check-ins can be faked quite easily. Even fixed QR codes open doors to different kinds of cheating methods.

A picture of a QR code gets passed around when someone skips class. One learner logs in using another's details just by showing the image. Someone else marks presence for a friend who never shows up. Scanning begins even if the real person is miles away. Attendance registers tick names that aren't really there. A phone display fools the system while the actual student stays home.

Later on, someone might grab a working QR code then send it again without changes. A hacker could copy the data once it's sent, using old info like new. Someone changes the check-in records on purpose. A hacker swaps what shows up when you scan a code. Wrong info sneaks into the system through fake inputs. Altered timestamps slip past security checks. Fake entries appear without warning. Digital fingerprints get swapped overnight. Records shift even when no one touches them directly.

When attendance records aren't trustworthy, it weakens audits, messes up financial aid rules, also puts students at risk. A way to lock down each check-in instantly, so nobody can deny it later, isn't just useful — it's overdue.

2. Proposed Solution: A Unified Secure Attendance System

The proposed system is made up of one web app containing three distinct portals (Teacher, Student, Admin), supported by a unified Express.js API and Turso (libSQL) database that is distributed worldwide.

2.1 Core Architecture

- **Backend API:** The backend API will be developed in Node.js using Express (api/index.js), which will include logic related to authentication, QR code creation, session management, and taking attendance.
- **Frontend Portals:** A responsive user interface for each role-specific portal (/teacher, /student, /admin) built with glassmorphism UI design.
- **Database:** Turso (libSQL) provides an edge-optimized, ACID-compliant data store.

2.2 Operational Workflow

1. Teacher signs into the application, creates a class, and begins an attendance session.
2. Server automatically generates a QR code which is updated every 15 seconds (QR WINDOW SECONDS), containing a newly generated unique cryptographic nonce and HMAC.
3. Student uses their phone to scan the QR code. Client device captures a unique device fingerprint and sends QR code contents to the server.
4. Server undergoes a 8 step authentication process before taking attendance.
5. Teacher is able to see attendance status in real time and lock the session.

3. Security Features and Threat Mitigation

The system's security is grounded in a multi-layered "defense-in-depth" strategy. A detailed STRIDE threat model was created to systematically identify and address risks.

3.1 Summary of Threat Model Findings 4.2 Core Security Mechanisms

1. **Dynamic, HMAC-Signed QR Codes:** Each QR packet {session id, course id, timestamp, nonce, signature} is signed using a QR HMAC SECRET generated on the server side. Any change in its content will cause the failure of signature validation, ensuring no modifications occur.
2. **Single-Use Nonces & Expiry:** The 15 seconds time period and the used nonces database table guarantee the single-use nature of the QR codes, making them resilient against the risk of sharing screenshots for replay purposes.
3. **Device Fingerprinting & Binding:** A hash value of a client-side fingerprint (screen resolution, timezone, canvas data, and other properties) is generated. Upon the first successful QR scan, the client is linked with the student's account. Only one client is allowed hence eliminating the threat of proxy participation.
4. **JWT-Based Authentication:** Sessions are managed using JSON Web Tokens stored in HTTP-only, SameSite=Strict cookies, making them immune to XSS-based token theft.

Table 1: STRIDE Threat Analysis Summary

STRIDE Category	Key Threat	Primary Mitigation
Spoofing	Impersonation via credential theft or device sharing	bcrypt password hashing, JWT, and Device Fingerprint Binding on first scan
Tampering	Modification of QR data or attendance records	HMAC-SHA256 signatures on QR payloads; session locking mechanism
Repudiation	Student denies marking attendance	Complete audit trail with timestamps, device IDs, and nonces in database
Info. Disclosure	Leakage of student data or tokens	RBAC, HTTP-only cookies for JWT storage, and enforced HTTPS
Denial of Service	Flooding system with fake QR generation requests	Rate limiting on all endpoints; authentication required for session creation
Elevation of Privilege	Student accessing teacher endpoints	Role-Based Access Control (RBAC) with server-side role verification from DB

4. Security Testing and Results

A comprehensive testing strategy was employed to validate the system's security posture, as detailed in the Secure Code Review.docx and Secure Testing Techniques.docx files.

4.1 Static Analysis & Code Review

- **Bandit (Python) & ESLint (Node.js):** This involved analyzing the code for typical security weaknesses. The only security issue found in the code with confidence was the possibility of being bound to all interfaces, i.e., 0.0.0.0, which is fine for test runs but should be reviewed before going live.
- **Manual Secure Code Review:** After reviewing each line of critical code files (auth.py, routes.py, index.js), there were no vulnerabilities such as SQL injections, hard-coded secrets, or logical errors since parameters were used.
- **Complexity & Maintainability (Radon):** The mark attendance feature was identified as having a high degree of complexity (lots of locals and returns) since it involves an 8-step pipeline.

4.2 Dynamic Testing & Penetration Simulation 5.3 Dependency Scanning (Safety & npm audit)

Table 2: Penetration Test Results

Test Technique	Objective	Result
Replay Attack	Resubmit a captured QR payload	Mitigated. All attempts received 409 Conflict (nonce used) or 410 Gone (expired)
QR Tampering	Modify a field (e.g., timestamp) in a valid QR payload	Mitigated. Server responded with 403 Forbidden (signature verification failed)
Auth. Bypass	Access teacher API endpoint with a student JWT	Mitigated. Server responded with 403 Access Denied due to RBAC enforcement
Privilege Escalation	Modify JWT payload (e.g., change role from 'student' to 'teacher')	Mitigated. Server-side role is fetched from the database, not trusted from the JWT
SQL Injection	Submit ' OR '1'='1 as a university .id	Mitigated. Parameterized queries (SQLAlchemy) treated input as data, not code
Device Mismatch	Attempt to mark attendance from a different device after binding	Mitigated. Server responded with 403 Device mismatch

- **Safety (safety report.txt):** Safety scan revealed that there were 139 vulnerabilities in 48 packages within the python ecosystem. This is mainly attributed to the development/analysis environment, Kali Linux, and its indirect dependencies like mitmproxy and pillow. The dependencies for the project itself (Flask, PyJWT, cryptography) have been secured.
- **npm Audit:** A standard npm audit would be run pre-deployment to ensure Node.js dependencies are also secure.

4.4 OWASP ZAP Dynamic Scan

The automated active scanning in OWASP ZAP failed to detect any critical vulnerabilities in the system. The medium and low-level vulnerabilities found during testing were mostly due to configuration errors.

- **Medium Risk Alerts (5):** CSP and HSTS security headers are missing or incomplete.
 - **Remediation:** This issue has been rectified as CSP and HSTS headers
- **Informational Alerts (3):** Information leakage (comments in JS file), and cross-domain JavaScript source inclusion (Google Fonts, QR library).
 - **Remediation:** The comments are classified as a low vulnerability, while the QR library is an essential external dependency.

5. Risk Management Summary

As per the Risk Management Document.pdf, the project effectively managed to reduce the overall risk level from High (70.6%) to Low (19.4%).

- **Critical Risks (1):** QR Code Replay Attack → Managed by nonces, time window management, and HMAC.
- **High Risks (7):** Authentication bypassing, data manipulation, server outage → Managed by JWT, digital signature usage, and server redundancy planning.
- **Remaining Risks:** The only remaining risk which has been accepted is zero-day vulnerability on the third-party library used in the software application.

6. Conclusion

Indeed, this particular project has been successful in achieving its key goal of avoiding proxy attendance. This is made possible through the security-in-depth concept that uses a number of security measures that are independent from each other to defend the system from any attacks.

Testing for various issues such as threat modeling and dynamic penetration testing shows how effective some of these measures have been. Although there was need to make some adjustments on the security headers and dependency management, the actual mechanism used to mark the attendance is not vulnerable to attacks such as replay and privilege escalation attacks.